MPSTME New Canteen

Full Project Report & Viva Preparation Guide

Fresh food, minus the queue.

Prepared for	MPSTME project review and viva examination
Canteen operator	Laxmi Caterers
Technology	HTML, CSS, JavaScript, Node.js, SQLite
Live application	https://mpstme-canteen.onrender.com
GitHub	https://github.com/saai2005/CANTEEN.SYSTEM
Role-page release	de879b8 — Add dedicated canteen role pages

What this report contains

A plain-language guide to the three user pages, order workflow, server and database, counter-based neural wait predictor, security and deployment limitations, validation results, and viva questions with suggested answers.

Prepared: 30 October 2026

Contents

1. Executive summary 4

2. Problem statement 4

3. Goals and scope 4

Goals . . . 4

User roles . . . 4

4. System architecture 5

High-level components . . . 5

Front end . . . 5

Back end . . . 5

Database . . . 5

5. Main files 6

6. Menu and counter model 6

Five preparation counters . . . 6

7. End-to-end operating pipeline 7

A. Startup . . . 7

B. Customer ordering . . . 7

C. Kitchen processing . . . 7

D. Cash-desk payment and handover . . . 7

E. Customer history and admin ledger . . . 8

8. Neural-network wait-time estimation 8

Neural-network architecture and training . . . 8

What happens inside the model . . . 9

End-to-end learning and prediction lifecycle . . . 9

How to evaluate it responsibly . . . 10

Preparation work for one counter task . . . 10

Cold-start queue estimate . . . 10

Shared cross-customer queue . . . 10

Model and estimate limitations to explain in a viva . . . 11

9. Order and payment states 11

Order states . . . 11

Payment states . . . 11

10. API overview 11

11. Authentication, privacy, and security 12

Deployment caveats . . . 12

12. Data integrity, backups, and retention **12**

13. Running and demonstrating the project **13**

Requirements . . . 13
Start the server . . . 13
Render deployment . . . 13
Sign-in and input formats . . . 13
First administrator . . . 13
Test from phones on the same Wi-Fi . . . 13

14. Validation performed and remaining test work **14**

Checks already performed during development . . . 14
Recommended tests to add . . . 14

15. Current limitations and future improvements **14**

Future work . . . 15

16. Viva questions and suggested answers **15**

Project fundamentals . . . 15
Architecture and technology . . . 15
Orders and counter workflow . . . 16
Wait-time logic . . . 16
Further ML viva questions . . . 16
Backend, data, and security . . . 17
Limitations and evaluation . . . 17

17. Short presentation script **18**

1. Executive summary

The project digitizes food ordering and order preparation for a campus canteen. Students, faculty, and staff can browse the menu, place orders, select a demo online payment or cash-at-counter option, see preparation estimates, and follow order progress. Kitchen staff see incoming orders and counter-specific preparation queues. Cash-desk staff manage COD payments and handover. Administrators can review records, manage selected settings, and export reports.

The central operational idea is that food is prepared at five independent counters. A meal-counter queue should affect meal estimates, but should not make a drink-counter estimate longer. The system assigns each menu item to a counter and trains a separate neural-network wait predictor for each counter from actual completed-ticket data. During the per-counter cold start, it clearly falls back to the existing preparation/workload estimate.

When run using `npm start`, the app uses a Node.js server and SQLite database so its three role-specific pages share the same accounts and orders. Use the server URLs rather than opening an HTML file directly.

2. Problem statement

Campus canteens can experience queues, uncertain preparation times, order mix-ups between food sections, and difficulty coordinating kitchen progress with payment and collection. Paper or verbal workflows make it harder for customers to know when to collect food and harder for operators to keep an accurate order and payment history.

This project addresses those issues with a shared digital workflow:

1. A customer submits a menu-based order.
2. The kitchen reviews and accepts or rejects it.
3. Accepted items are assigned to their preparation counters.
4. Counter staff update preparation progress.
5. Cash-desk staff record COD payment where applicable.
6. The order can be handed over once it is ready and paid.
7. Customers and staff can review the order status and the accounting record.

3. Goals and scope

Goals

1. Provide a responsive menu and digital ordering experience.
1. Route items to five operational counters.
1. Estimate waiting time using the relevant counter's workload.
1. Keep order status and payment status visible to the appropriate user.
1. Maintain persistent order, payment, and administrative records in SQLite.
1. Support search, history, CSV export, receipt printing, and order archiving.
1. Make the app usable from more than one device on a trusted local network.

User roles

Role	Main responsibilities
Student	Browse menu, place orders, view own live orders and history, reorder, and hide completed orders from their view
Faculty / staff customer	Same customer ordering and history features
Kitchen	Review incoming orders, accept or reject them, and update food preparation status

Role	Main responsibilities
Cashier	Find COD orders, record payment, undo an eligible payment, cancel orders, and hand over eligible orders
Admin	Manage staff accounts and settings, view the accounting ledger, inspect audit history, export data, and archive/restore records

Customer accounts can self-register as student, faculty, or staff. Worker accounts are created by an administrator. The server-backed version requires a signed-in account; customer history is scoped to the current account.

4. System architecture

High-level components

```
student.html ■■
kitchen.html ■■■■ shared index.html app ■■ HTTP requests/polling ■■ Node.js server ■■ SQLite
cashier.html ■■                                                  ■■■ serves page shells/assets
```

Front end

1. `student.html`, `kitchen.html`, and `cashier.html` are separate page entry points for the student, kitchen, and cash-desk workflows. Each loads the same `index.html` application in its role-specific mode so the existing UI and shared functionality stay consistent.
1. `index.html` contains the shared user interface, styles, menu catalog, role-scoped application logic, and browser-side API integration.
1. The UI is built with standard HTML, CSS, and browser JavaScript; there is no separate front-end framework or bundler.
1. The student page presents the menu, cart, checkout, queue estimates, and customer history. The kitchen page presents incoming orders and preparation counters. The cash-desk page presents COD payments and handover. Admin controls remain in the combined root interface.
1. `student.html`, `kitchen.html`, and `cashier.html` are lightweight full-window HTML shells that load the shared app with a role-specific mode. They are separate entry pages, not three duplicated copies of the app.
1. The role pages load the shared app in a full-window iframe. This keeps the original styling and behavior consistent while giving each role a separate URL.
1. Role-specific pages filter navigation and views, while the Node.js server independently enforces permissions for every protected API action.
1. In server mode, the browser calls the same-origin JSON API and periodically checks for updated data.
1. In local-file demo mode, the app uses browser storage. That mode does not provide authenticated, cross-device records.

Back end

1. `server.js` uses Node.js built-in `http`, `crypto`, `fs`, `path`, and `node:sqlite` modules.
1. It serves the front end and implements JSON endpoints for accounts, orders, queue estimates, payments/audits, admin history, exports, and settings.
1. Order input is checked on the server against the menu catalog and saved settings; the server does not rely on the submitted customer price as authoritative.
1. The application uses HTTP polling, not WebSockets or a hosted realtime service.

Database

The database is created locally at `data/canteen.sqlite` by default. SQLite uses write-ahead logging and foreign-key enforcement. The main tables are:

Table	Purpose
`users`	User identity, role, password salt/hash, creation time, and disabled flag
`sessions`	Hash of the session token, its user, and expiration
`orders`	Owner, creation/update timestamps, hidden/archive flags, and serialized order details
`payment_logs`	Payment, undo, refund, or adjustment events, with method, amount, actor, time, and reason
`admin_action_logs`	Administrative and order-change audit entries
`settings`	Saved app settings as JSON

Order details are stored as JSON in the `orders.data_json` field. The payment and admin audit events are stored in dedicated tables so they can be independently queried and retained. Indexes support customer history and time-ordered lookups.

5. Main files

1. student.html — student-facing page entry point.
1. kitchen.html — kitchen page entry point.
1. cashier.html — cash-desk page entry point.
1. index.html — shared UI, menu catalog, application logic, and browser-side API integration.
1. server.js — HTTP API, SQLite schema, authentication, authorization, catalog validation, order-state checks, payment and admin auditing, exports, backup, and retention.
1. PROJECT_REPORT.pdf — formatted project report and viva preparation guide.
1. package.json — Node.js engine requirement and `start`/`test` scripts. The project currently has no npm runtime dependencies.
1. README.md — local startup, account setup, network access, data location, backup, and payment limitations.
1. Laxmi Caterers Menu New MPSTME.pdf — supplied menu reference.

6. Menu and counter model

The server loads the menu catalog from the JavaScript menu definitions in `index.html`. The current catalog contains **272 menu entries** across **18 populated categories**:

Breakfast, Chaat, Chinese, Chinese Starters, Egg, Franky, Hot Beverages, Juices, Lassi, Lunch, Maggi, North Indian, Pasta & Fries, Pav Bhaji, Pizza, Sandwiches, Shakes, and South Indian.

The UI also includes a Fruits category/filter, but the current catalog contains no fruit items; the app displays a message that no fruit items are listed. Before presenting the menu as a final transcription of the supplied PDF, compare every name and rupee price against the PDF and correct any discrepancies.

Five preparation counters

Counter	Intended menu group
1. Meals	Lunch, thali, North Indian, breads, rice, and meal dishes
2. South Indian & Breakfast	Dosa, idli, uttapam, and breakfast items
3. Snacks & Chaat	Pav bhaji, chaat, franky, sandwiches, and egg items
4. Chinese	Chinese starters, fried rice, and noodles

Counter	Intended menu group
5. Beverages & Fruit	Lassi, juices, shakes, hot beverages, and fruit items if present

The catalog assigns a default counter to each item. Admin settings can change an item's counter mapping; the saved mapping is used when new orders are validated. Counter names, staffing, item preparation time, and menu price also have settings in the UI.

7. End-to-end operating pipeline

A. Startup

1. `npm start` starts `server.js`.
2. The server creates/opens the SQLite database and its tables.
3. It runs the daily backup and retention routines.
4. The HTTP server listens on the configured host and port (defaults: `127.0.0.1:4173`).
5. A browser opens `http://localhost:4173`; other devices can use the computer's LAN IP when the server is configured to listen on the LAN.

B. Customer ordering

1. A student, faculty member, or staff member signs in or registers.
2. The customer searches or filters the menu and adds quantities to the cart.
3. The app groups cart items by preparation counter and displays estimates.
4. Checkout collects name/role and payment choice: COD, UPI demo, or card demo.
5. The client sends the order to `POST /api/orders`.
6. The server assigns the order number and checks that:
    1. menu IDs exist;
    1. quantity and payment method are valid;
    1. the service is open according to the saved schedule and server clock;
    1. the item/category/counter is available;
    1. the price, preparation time, and counter are loaded from the server-side catalog/settings.
    1. A valid order is stored with status `incoming` and appears on the staff order board.

C. Kitchen processing

1. The kitchen board receives shared order data through periodic server polling.
2. A worker accepts or rejects an incoming order.
3. If the policy requires cash-desk payment before preparation, a COD order can wait in `awaiting-payment`; otherwise, acceptance dispatches it to counters.
4. Dispatch groups the order's items by counter and creates a task for each counter.
5. Each counter queue is ordered by acceptance time and queue sequence (FIFO).
6. Counter staff mark tasks as preparing/ready. The full order becomes ready when all involved counter tasks are ready.
7. Rejection, cancellation, and readiness are reflected in the customer's order view when it next polls for updates.

D. Cash-desk payment and handover

1. Cashier searches for the order in the cash-desk list.

2. For an unpaid COD order, cashier confirms the exact amount and selects cash or UPI at the desk.
3. The payment status and payment log are updated.
4. Payment undo is available only within the configured short window and is logged.
5. A ready-but-unpaid order is shown as payment pending.
6. Handover is allowed only if the order is both `ready` and `paid`.
7. The order is marked `collected` and remains in the persistent history.

E. Customer history and admin ledger

1. Students can see their own order history, receipt details, timestamps, status, and payment details.
1. A customer may hide completed orders from their own view; this is not a deletion of the accounting record.
1. Reordering adds the previous items to the current cart using current menu information.
1. Admins can search/filter the full ledger, inspect payment/action audit entries, export CSV, print/save views as PDF, and archive/restore orders.
1. CSV is intended to be opened in spreadsheet software such as Excel; the app does not generate a native `.xlsx` workbook.

8. Neural-network wait-time estimation

Queue prediction is the project's machine-learning component: it uses supervised learning with a feed-forward neural-network regressor. There is one independently trained model for each of the five counters, so a busy Meals queue does not directly inflate a Beverages prediction.

When a counter ticket is accepted, the server captures its feature values. When that ticket is marked ready, the server records one training example whose target is the elapsed time in minutes between server-recorded order acceptance and counter completion. Duplicate examples for the same order/counter are prevented. Previously completed tickets without captured features are not used as training examples.

The eight input features are:

1. Number of orders ahead at that counter.
2. Remaining queued workload in minutes.
3. This ticket's configured preparation estimate.
4. Quantity on the ticket.
5. Staff assigned to the counter.
6. Day/rush multiplier.
7. Hour in Asia/Kolkata.
8. Day of the week.

Feature values and target minutes are normalized, and target wait is bounded at 240 minutes.

Neural-network architecture and training

1. **Architecture:** 8 input values → one hidden layer of 8 ReLU neurons → one linear output predicting minutes.
1. **Objective/training:** mean-squared-error regression, trained with batch gradient descent for 280 epochs per training run.
1. **Training cadence:** after each newly completed counter ticket, the server retrains that counter's model using up to its latest 300 examples.
1. **Persistence:** SQLite stores training examples and model weights. Authenticated queue responses provide ready models to student browsers, which run inference with the same network structure.

1. **Activation/cold start:** a counter uses neural predictions after at least 8 completed examples. Before that, the UI labels the model as learning and uses the transparent preparation/workload fallback. A new database starts in this cold-start state.

This is a real, data-trained neural model, not an immediate prediction from a fresh database. Eight examples is a practical activation threshold, not proof of statistical accuracy.

What happens inside the model

Before training or prediction, each input is converted to a number between 0 and 1 by dividing by a configured scale and clipping to that range:

Input	Scale
Orders ahead	20 orders
Queued workload	240 minutes
Ticket preparation time	60 minutes
Ticket quantity	20 units
Counter staff	10 people
Rush factor	2.0
Local hour	23
Weekday index	6

For one example, the forward pass calculates each hidden neuron as `ReLU(b + weighted sum of the 8 inputs)`. ReLU outputs zero for a negative value and otherwise keeps the value. The output neuron takes a weighted sum of the 8 hidden activations and a bias. That normalized output is multiplied by 240 to get minutes and bounded to the range 0–240. Student estimates additionally use a five-minute minimum.

During learning, the model compares its normalized prediction with the recorded normalized target. The training loss is mean squared error: the average of `(prediction - actual)²` across the saved examples. Backpropagation computes how much each weight and bias contributed to that error; batch gradient descent adjusts them in the direction that reduces the loss. This implementation uses a learning rate of `0.035` for 280 passes (epochs) over the training set. Initialization is deterministic for a given counter and training-set size, which helps make runs repeatable.

In plain language, the model learns how combinations such as a larger queue, more workload, a larger ticket, different staffing, or a rush period have historically related to elapsed completion time. It does not contain a hand-written rule that guarantees each feature always changes the prediction in a particular direction.

End-to-end learning and prediction lifecycle

1. **Order accepted:** the server records the current counter workload, this ticket's prep time and quantity, staffing, rush factor, and IST calendar values on the counter task.
2. **Ticket completed:** when kitchen staff mark that task ready, the server calculates elapsed minutes from the order's accepted timestamp to that task's completion timestamp. This is the supervised label; it includes both queuing and preparation time.
3. **Training row stored:** feature values and actual elapsed minutes are inserted into SQLite once per order and counter. A unique database key prevents the same ticket from being added twice.
4. **Counter model retrained:** the newest 300 examples for that counter (or all available examples if fewer) are loaded and trained. Its weights and sample count are saved in SQLite.
5. **Customer estimate:** authenticated queue data supplies workload counts without revealing other customers' identities. The browser combines this with the customer's queue, normalizes the current features, and runs a forward pass using that counter's saved model.

6 **Order ETA:** the app predicts each involved counter independently. Since counters can work in parallel, the order ETA is the maximum of those counter estimates, not their sum.

An example sequence illustrates the data flow without claiming a particular prediction: if a ticket at the Meals counter is accepted with a queue of 3 orders and later takes 18 minutes until marked ready, its captured features and `18`-minute target become a Meals training example. That row does not train the Beverages model.

How to evaluate it responsibly

The current implementation does **not** reserve a test set, report prediction accuracy, tune hyperparameters, or compare the neural network against a measured baseline. Its passing unit tests show that feature normalization works, the activation threshold is honored, and predictions are finite and respond to a workload pattern in synthetic test data; they do not prove accuracy on real canteen orders.

For a credible evaluation, collect enough representative real tickets, train using earlier tickets, and evaluate on later tickets that were not used for training. Report:

1 **MAE (mean absolute error):** average absolute difference between predicted and actual minutes; easy to interpret in minutes.

1 **RMSE (root mean squared error):** square root of the average squared error; larger misses count more heavily.

1 **Baseline comparison:** calculate the same metrics for the cold-start estimate and compare against the neural model on the same held-out tickets.

1 **Per-counter results:** report sample counts and errors separately for each counter, and consider different rush modes and times of day.

With only the eight-example activation threshold, these metrics would be highly uncertain. Gather substantially more data, check for missing or unusual timestamps, and monitor errors over time before relying on model estimates operationally.

Preparation work for one counter task

The client groups items by counter. The task preparation estimate is:

```
task base work =
    longest configured item preparation time
    + one minute for each additional unit beyond the first
```

For example, one roti item configured for 5 minutes and one sabji item configured for 15 minutes produce a task estimate of `15 + 1 = 16` minutes in the current grouping implementation. This is **not** the “5 + 15 = 20 minutes” example from the original requirement. If the intended model is to sum different item preparation times rather than use the longest item time, the formula needs to be changed and tested.

Cold-start queue estimate

1 Queue work is independent per counter.

1 The selected rush factor is Quiet `1.0×`, Normal `1.25×`, Busy `1.5×`, or Rush `2.0×`.

1 The estimate accounts for staff/station count and includes a small handover/packing buffer.

1 The displayed order total is based on the **maximum** estimated wait among counters involved, because those counters can work in parallel.

1 A counter with no queued work still uses a minimum estimate (approximately five minutes in the student counter strip).

Shared cross-customer queue

Authenticated customer clients periodically call `GET /api/queue`. The response contains only per-counter counts and remaining preparation workload for **other customers' accepted and preparing orders**; it does not contain customer names, item names, or order numbers. The client combines this aggregate with the signed-in customer's own active orders.

The endpoint refreshes on an approximately five-second interval. The rest of the app also polls server state. This is near-real-time polling, not a WebSocket push stream.

Model and estimate limitations to explain in a viva

1. Incoming orders that have not yet been accepted are intentionally excluded from the cross-customer aggregate, because they have not been committed to a kitchen queue yet.
1. The aggregate has a fixed five-counter structure and estimates work from task preparation times and server settings.
1. Models learn only from completed tickets captured after this feature was introduced; a fresh installation needs real kitchen completions before it can provide neural estimates.
1. There is no held-out validation set or displayed accuracy metric yet. Measure MAE/RMSE on later completed tickets before making accuracy or production-readiness claims.
1. Queue/staff scheduling is an approximation; models do not capture every operational detail such as cooks specializing in dishes, parallel stations with different speeds, or pauses.
1. The cold-start formula uses the longest preparation time among items at one counter plus one minute per extra unit. It should be aligned with the final agreed food-timing rule before claiming that it implements every original worked example.

9. Order and payment states

Order states

The main workflow is:

```
Placed / Incoming → Accepted → Preparing → Ready → Collected
                          ■ Rejected
Any eligible active order may also be cancelled with a reason.
```

For COD-before-preparation configuration, acceptance can first set `awaiting-payment`; recording payment dispatches the order to the kitchen.

Payment states

1. `unpaid`: normally a COD order awaiting payment.
1. `paid`: a COD payment recorded at the desk or an online demo checkout.
1. `refunded`: a paid order later refunded/cancelled through supported application flows.

The user interface prevents normal handover until an order is both ready and paid. Status transitions are checked by the server for kitchen and cashier roles; the admin has broader access.

10. API overview

The server exposes JSON routes under `/api/`. Important routes include:

Route	Purpose
`GET /api/health`	Basic app/SQLite health check
`POST /api/auth/register`	Customer registration for student/faculty/staff roles
`POST /api/auth/login`	Sign in and create a server session
`POST /api/auth/logout`	End the current server session
`GET /api/auth/me`	Return the current authenticated user
`GET /api/queue`	Return privacy-safe aggregate queue counts/work per counter
`GET /api/orders`	List permitted order history

Route	Purpose
`POST /api/orders`	Validate and place a new order
`POST /api/orders/:id/hide`	Hide one completed order from its owner's history
`POST /api/orders/hide-all`	Hide completed orders from the customer's own view
`GET /api/state` / `PUT /api/state`	Load/save role-scoped app/order state
`GET /api/desk/audit`	Read payment events for permitted desk roles
`GET /api/admin/orders`	Search/filter/paginate admin ledger
`GET /api/admin/audit`	Read admin action and payment audit records
`POST /api/admin/orders/:id/archive` or `/restore`	Archive or restore an order
`POST /api/admin/archive-range`	Bulk archive orders older than a chosen date
`GET /api/admin/export.csv`	Export accounting records as CSV
`GET/POST /api/admin/users`	List/create worker and admin accounts

Role checks are applied in the server routes; hiding a UI control is not the security boundary.

11. Authentication, privacy, and security

1. Passwords are stored as salted scrypt-derived hashes, not plaintext.
1. A random session token is sent in an HTTP-only, SameSite cookie; the database stores a hash of the token and its expiry.
1. Customer order queries are limited to that customer's own account.
1. Kitchen and cash-desk/admin routes require an authenticated account with an allowed role.
1. The cross-customer queue response is aggregated and excludes direct customer/order identifiers.
1. Server order validation checks availability, catalog items, payment type, and quantities. Opening/closing-time restrictions are currently disabled, so orders can be placed at any time.
1. Payment and significant order/admin changes are represented in audit records.

Deployment caveats

The current server uses plain HTTP, and session cookies do not set the `Secure` attribute. The README therefore advises against exposing this development server directly to the public internet. Before public production use, deploy behind HTTPS with a production reverse proxy and review CSRF protections, rate limiting, session policy, monitoring, backups, privacy requirements, and payment-provider integration.

12. Data integrity, backups, and retention

1. SQLite is the local persistent data store.
1. The server makes daily database copies under `data/backups/` and keeps up to 30 dated backup files.
1. The admin can archive orders without permanently deleting the accounting record.
1. The optional retention setting can automatically archive records older than one or three years; “keep forever” is the default.
1. Students hiding history only changes visibility for that customer; it does not remove the admin's accounting record.
1. Payment records are append-style audit events; corrections should be represented as undo/refund/adjustment events rather than silently erasing payment history.

The daily backup is stored on the same machine as the database. Important records should also be backed up to a separate, access-controlled location.

13. Running and demonstrating the project

Requirements

1. Node.js `>=22.5.0` (Node.js 24 is recommended by the README).
1. No npm packages are currently required.

Start the server

```
cd C:\Users\HP\OneDrive\Desktop\canteen
npm start
```

Open `http://localhost:4173/student.html`, `http://localhost:4173/kitchen.html`, or `http://localhost:4173/cashier.html`. Open the root URL for the combined Admin interface. Do not open an HTML file directly if you need accounts or shared server state.

Render deployment

The Render Blueprint starts the same Node.js server using `npm start`. The public service is `https://mpstme-canteen.onrender.com`; its three role routes are `/student.html`, `/kitchen.html`, and `/cashier.html`, and the root route remains the combined Admin interface. The live health route is `/api/health`. Release `de879b8` was pushed to the repository's `main` branch and verified as Live in Render; all three role routes returned HTTP 200.

The configured free plan stores SQLite data under `/tmp/canteen-data`. This is temporary storage: accounts, orders, and backups can be lost when the service sleeps, restarts, or redeploys. The current free deployment is suitable for a demonstration, not for retaining important production records.

Sign-in and input formats

1. The username field gives an example. Usernames are 3–80 characters and may contain letters, numbers, underscores, dots, `@`, plus signs, or hyphens.
1. New ordinary accounts require a password of 10–200 characters. A Render-configured demo account may use its specific configured minimum. Sign-in accepts 1–200 characters because it must allow existing credentials; the username and password still have to match a real account.
1. The example text is a hint, not an account-creation bypass. The server verifies credentials and applies account-role permissions.
1. At demo checkout, any non-empty UPI ID up to 80 characters is accepted. It is not verified, used to start a payment, or saved as a payment credential. No real money is charged.

First administrator

If no admin account has been created yet, run:

```
node server.js --create-admin admin "Canteen Admin"
```

Enter a password of at least 10 characters. On an existing installation, use the existing admin account rather than trying to create a duplicate administrator. The admin can create worker accounts; customers can self-register.

Test from phones on the same Wi-Fi

1. Start the server with `HOST=0.0.0.0`.
2. Find the computer's LAN IP using `ipconfig`.
3. On a device on the same trusted Wi-Fi, open `http://<computer-LAN-IP>:4173`.
4. If Windows Firewall blocks the connection, allow the Node.js server on the private network.
5. Do not expose this development HTTP service to the public internet.

14. Validation performed and remaining test work

Checks already performed during development

1. Backend JavaScript syntax check: passed.
1. Front-end inline JavaScript syntax check: passed.
1. `npm test`: passed, including three queue neural-network unit tests for feature normalization, cold-start activation threshold, and workload-sensitive predictions.
1. Neural queue API integration: completed eight counter tickets through the authenticated customer/kitchen API workflow; verified the counter model activated with eight saved samples, was reported as active to the admin, and completed tickets no longer contributed to live queue load.
1. API smoke check with two student accounts: a customer saw another customer's Meals queue count/workload, did not see the other account's own queue in the cross-customer aggregate, saw other counters remain independent, and received no customer name or order number from the queue API.
1. Browser smoke check: the signed-in student UI displayed the shared Meals workload and adjusted its wait estimate.
1. Page smoke check: all three dedicated HTML routes and `/api/health` returned HTTP 200 on an isolated local Node.js server.
1. Deployment check: Render showed release `de879b8` as Live; the public student, kitchen, and cash-desk pages returned HTTP 200 and `/api/health` reported `{ "ok": true, "sqlite": true }`.
1. Earlier API/browser checks covered student history privacy, admin ledger rendering, order status and payment flows; these checks are not a substitute for a maintained automated regression suite.

Recommended tests to add

1. Two-customer queue aggregation and own-order exclusion.
2. Independent counter queues and correct rush/staff calculations.
3. Menu item/price/availability validation against tampered client requests.
4. Order placement behavior with opening/closing-time restrictions disabled.
5. Valid and invalid kitchen state transitions and FIFO ticket order.
6. COD amount confirmation, duplicate-payment rejection, undo window, and audit entries.
7. Ready-but-unpaid handover rejection.
8. Student history ownership, hide/clear behavior, pagination, archive/restore, and export.
9. Browser tests at mobile and tablet widths.
10. Backup restore procedure, not only backup-file creation.

15. Current limitations and future improvements

These are important to state accurately during a project review:

1. **Online payment is a simulation.** UPI and card do not charge money; the application does not collect card details.
2. **Realtime is polling.** It is not implemented with WebSockets or a hosted realtime database.
3. **Menu verification matters.** The current code contains 272 entries and an empty Fruits filter. Compare every menu item/price with the supplied PDF before making a claim of exact transcription.
4. **Preparation formula needs product-owner confirmation.** Current code uses the longest configured item prep time plus one minute per additional unit. The original example of roti 5 + sabji 15 → 20 minutes instead implies summing dissimilar item times. Choose one rule and test it.

5. **Neural queue prediction is an early prototype.** It needs at least eight completed examples per counter before activation. It has no held-out accuracy evaluation and cannot model every real kitchen station/cook constraint.
6. **Automated test coverage is limited.** The current three unit tests exercise the neural model, but most API workflows do not yet have automated regression coverage.
7. **Deployment security needs further hardening.** Render serves the public site over HTTPS, but the application cookie does not currently set the `Secure` attribute, and additional protections such as rate limiting, CSRF defenses, and monitoring should be considered before production use.
8. **Backups are local.** Use an off-device backup and practice restoring it.
9. **Some analytics are derived from currently available order/task records.** Treat them as operational estimates until completeness and reconciliation are validated with real canteen use.
10. **Customer login is required in server mode.** The frontend includes sign-in/register controls so the server can enforce account ownership and preserve private order history.

Future work

1. Add a real payment provider and reconcile signed provider webhooks server-side.
1. Add WebSocket/SSE delivery or a robust managed realtime service if polling is insufficient.
1. Evaluate neural predictions on held-out real orders and add automated API/state-transition and browser-based critical workflow tests.
1. Validate the complete menu against the source PDF and add explicit menu-import/versioning support.
1. Agree, implement, and test the preparation-time formula with actual kitchen staff.
1. Add operational dashboards based on validated timestamps and measured preparation data.
1. Add production-grade HTTPS deployment, rate limiting, CSRF defenses, account recovery, monitoring, and off-device backups.

16. Viva questions and suggested answers

Project fundamentals

1. What is the title of your project? MPSTME New Canteen Ordering System, a paperless campus food-ordering and kitchen-management application for Laxmi Caterers.

2. What problem does the project solve? It gives customers a way to order and track food digitally, routes accepted items to the appropriate counters, gives workers shared order visibility, and records order/payment history.

3. Who are the stakeholders? Students, faculty, campus staff, kitchen workers, cashiers, and canteen administrators.

4. What are the main functional modules? Customer menu/cart/checkout, customer order history, kitchen queues, cash-desk payments/handover, admin settings and account management, audit/history exports, and server/database services.

5. What is the main benefit over a paper workflow? Orders and their status are shared and searchable, customers can see progress, and the canteen has a persistent accounting/audit record instead of relying only on paper slips.

Architecture and technology

6. What technologies are used? HTML, CSS, and browser JavaScript on the frontend; Node.js built-in HTTP and SQLite APIs on the backend; SQLite for persistence.

7. Why are there separate HTML pages for each role? The student, kitchen, and cash-desk workflows have distinct entry URLs and focused interfaces. Each page shell loads the common UI/application in a role-specific mode so orders, permissions, styling, and behavior stay connected without duplicating core logic.

8. Why use SQLite? It is a lightweight relational database stored in a local file, requires no separate database server for this prototype, supports transactions, and is suitable for a local canteen demonstration.

9. Where is the menu stored? The current menu definitions are embedded in `index.html`. The server loads the catalog from those definitions and uses it to validate item IDs and prices.

10. How does a phone connect to the app? The Node server must listen on the computer's LAN interface, the phone must be on the same trusted Wi-Fi, and the phone opens the computer's LAN IP and port.

Orders and counter workflow

11. Walk through an order from placement to collection. A customer submits a cart; the server validates and stores it as incoming; kitchen accepts it and dispatches tasks by counter; counter staff mark tasks ready; cashier records COD payment if needed; the cash desk hands over only when the order is ready and paid; the order becomes collected.

12. Why are there five counters? The menu is divided into operational groups so each order item reaches the relevant preparation area and queue estimates remain independent by station.

13. What does FIFO mean in this system? First In, First Out: accepted tickets are ordered by acceptance timestamp and queue sequence, so earlier accepted work is presented ahead of later work.

14. How does an order with food and a drink get processed? The order is divided into counter tasks. For example, a meal can go to Meals and a shake to Beverages; the overall order is ready when all its required counter tasks are ready.

15. Can an unpaid COD order be handed over? No. The UI requires both Ready and Paid before displaying the handover action, and the server validates role/state transitions.

16. What happens when a counter rejects an order? The order is marked rejected and the customer can see its final status/reason. A previously paid demo order may be marked refunded through the supported flow and audit record.

Wait-time logic

17. What machine-learning topic is used in this project? Supervised learning with a feed-forward neural-network regressor. It predicts counter-ticket completion time in minutes from completed kitchen tickets.

18. Why is queue prediction a regression problem? The output is a continuous numeric value—predicted wait time in minutes—not a category or class.

19. What is the neural-network architecture? There is a separate model per counter, with eight input features, one hidden layer of eight ReLU neurons, and one linear output for wait time.

20. What are the model inputs and training target? Inputs are queue order count, queued workload, ticket prep time, quantity, counter staffing, rush factor, local hour, and weekday. The target is actual elapsed time from order acceptance until that counter ticket is marked ready.

21. How is the model trained and stored? After a counter ticket is completed, the server saves its features and actual elapsed time in SQLite and retrains that counter's model using up to 300 recent examples. The trained weights are also persisted in SQLite.

22. What happens before a counter model has enough data, and is the ETA exact? Neural estimates activate after eight completed examples per counter. Before then, the UI labels that counter as learning and uses the transparent preparation/workload fallback. Eight examples is not an accuracy guarantee, and actual kitchen delays can still change any ETA.

Further ML viva questions

How does the network learn from an order? The server stores the input conditions at acceptance and the actual elapsed time when that counter ticket becomes ready. It normalizes the data, runs a forward pass, measures squared prediction error, and uses backpropagation with gradient descent to update the network's weights and biases.

What does ReLU do here? ReLU is the hidden-layer activation: it returns `max(0, x)`. It lets the network model nonlinear relationships rather than only one straight-line formula.

Why normalize the input features? Queue counts, minutes, quantity, and hour use different numeric scales. Scaling and clipping them to 0–1 keeps their ranges comparable and makes optimization more stable.

Why use separate models for each counter? Each counter has different food, workflow, and preparation patterns. Training separately helps keep those patterns independent, so a Meals queue does not become Beverages training data.

Does the model guarantee that more queue always means more predicted wait? No. The network learns patterns from examples, and this implementation does not enforce monotonic behavior. Predictions should be checked on representative held-out data and monitored.

How would you detect overfitting or prove accuracy? Evaluate on newer tickets held out from training, compare MAE/RMSE with the cold-start baseline, and repeat by counter and operating condition. Do not use the training error alone as evidence of accuracy.

Is this a large language model, deep learning service, or classification model? No. It is a small feed-forward neural network implemented in JavaScript for supervised numerical regression. It predicts minutes; it does not generate text, classify orders, or call an external AI service.

Backend, data, and security

23. How are order records kept private? Customers authenticate with an account, and customer order queries are scoped to that account. Server route authorization is the primary access check; the interface alone is not trusted.

24. How are passwords stored? The server stores a salted scrypt-derived hash instead of plaintext and checks the submitted password against the stored hash.

25. What is stored in the database? Accounts, sessions, order records, payment logs, admin action logs, and saved settings. Order payload fields are currently serialized as JSON inside an SQLite order row.

26. How is order data validated? The server checks menu item IDs, quantity bounds, payment type, availability, and server-side catalog prices/settings before accepting a new order. Opening/closing-time restrictions are currently disabled.

27. How do customers hide history? Hiding sets a customer-visibility flag for completed orders. It does not delete the underlying order from the administrator's accounting ledger.

28. What is the purpose of the audit log? It records who performed important payment or administrative/order actions and when, which supports traceability and accounting review.

29. How are backups handled? The server creates a daily SQLite backup file and retains up to 30 dated backup files locally. A separate off-device backup is still recommended.

Limitations and evaluation

30. Are UPI and card payments real? No. They are demonstration flows. No payment provider is integrated and the app does not collect card numbers.

31. Does the app use WebSockets? No. It uses periodic HTTP polling (about every five seconds for shared queue workload and state updates) to refresh screens.

32. What does `npm test` currently prove? It runs three neural model unit tests. They check feature normalization, the activation threshold, and workload-sensitive predictions; they do not prove all application workflows are covered.

33. What is the biggest deployment concern? The public Render URL uses HTTPS, but the app still needs stronger production controls. For example, secure-cookie settings, rate limiting, CSRF protections, monitoring, and durable backups should be reviewed before handling real payments or important long-term records.

34. What would you improve next? Add automated API/browser tests, verify every catalog entry against the menu PDF, agree on the prep-time formula with canteen staff, integrate a payment provider, improve realtime delivery, and add production deployment security.

35. How would you measure whether the system is successful? Measure order processing time, estimate accuracy, average queue wait, rejection/cancellation rate, payment reconciliation errors, customer usability feedback, and successful order handover rate.

17. Short presentation script

> *Our project is a digital ordering and canteen operations system for the MPSTME New Canteen, run by Laxmi Caterers. Customers browse a menu and place an order from a browser. The Node.js server validates each order and stores it in SQLite. The kitchen accepts orders and routes each item to one of five preparation counters. Each counter has a supervised neural-network model trained on actual completed tickets to predict its wait; during cold start, a transparent baseline estimate is used until enough examples are collected. The app combines parallel counter estimates using the slowest counter and shares privacy-safe queue workload with customers. Cashiers record COD payments and can hand over an order only after it is ready and paid. The system also preserves order history, payment logs, admin actions, and backups. Online payments and realtime updates are currently demonstrations, so production use would require a payment provider, stronger deployment security, held-out model evaluation, and broader automated tests.*